DATA 3230 Data Visualization Final Project

Introduction

This project analyzes monthly MTA major incidents in New York City and examines whether weather conditions help explain variation in incident counts. Major incidents are defined as incidents that delay 50 or more trains. The analysis begins with weather-only models, then improves the model by incorporating subway ridership as a measure of system usage, and finally evaluates whether logarithmic transformations provide a better representation of the relationship between incidents, weather, and ridership.

Load Packages

```r
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
library(readr)
library(ggplot2)
library(lubridate)
```

```
Attaching package: 'lubridate'
```

```
The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

```r
library(tidyr)
library(httr)
library(car)
```

```
Loading required package: carData


Attaching package: 'car'

The following object is masked from 'package:dplyr':

    recode
```

```r
library(ggdist)
```

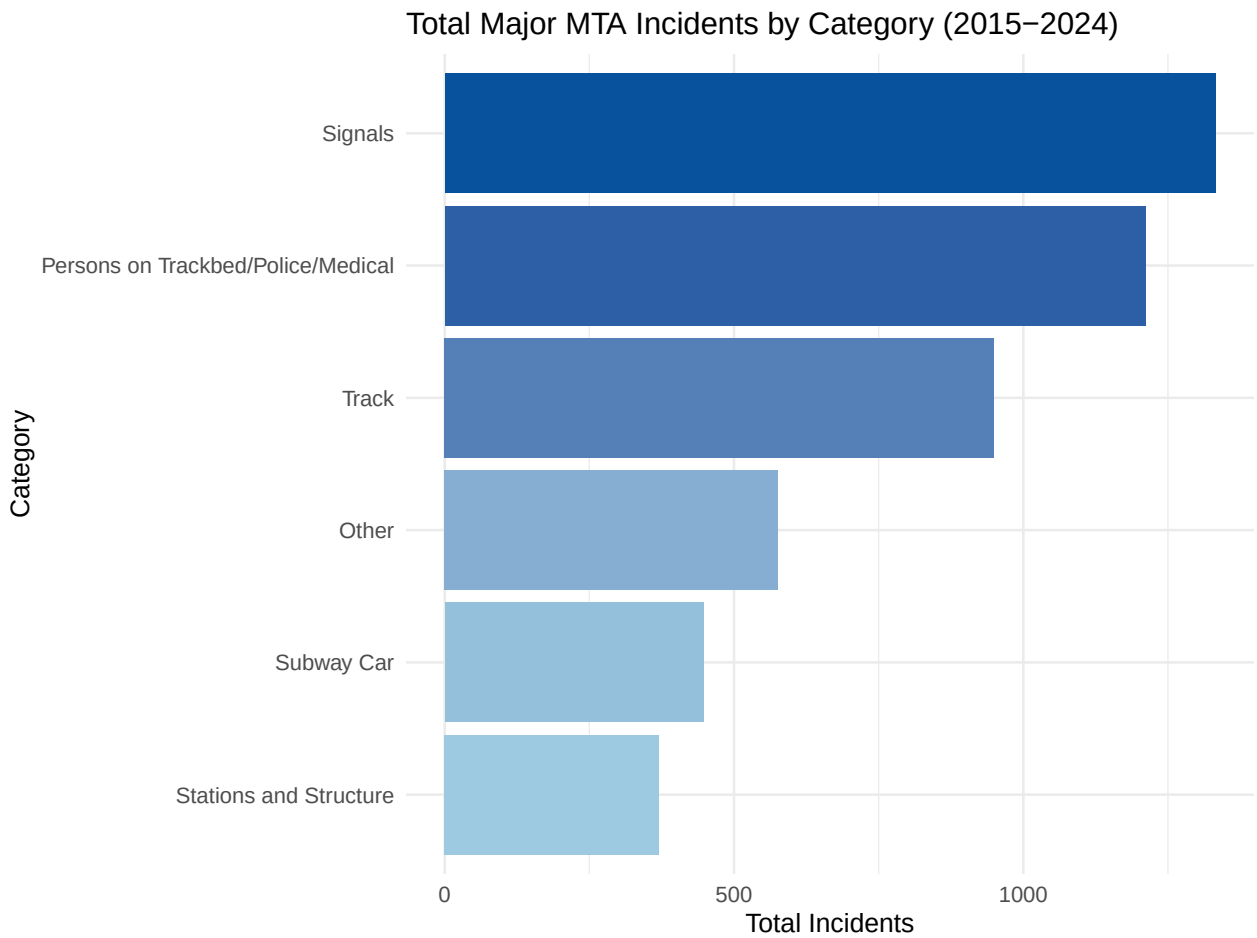
Load and Summarize Incident Data

```r
incidents_2015_2019 <- read_csv("~/Downloads/MTAIncidents 2015-2019.csv", show_col_types = FALS
incidents_2020_2024 <- read_csv("~/Downloads/MTAIncidents 2020-2024.csv", show_col_types = FALS

all_incidents <- bind_rows(incidents_2015_2019, incidents_2020_2024) %>%
  arrange(month)

summary_df <- all_incidents %>%
  group_by(month, category) %>%
  summarise(count = n(), .groups = "drop")

monthly_totals <- all_incidents %>%
  group_by(month) %>%
  summarise(total = n(), .groups = "drop") %>%
  arrange(month)

category_totals <- all_incidents %>%
  group_by(category) %>%
  summarise(total = n(), .groups = "drop") %>%
  arrange(desc(total))
```

Exploratory Visualizations of Incident Data

Total Incidents by Category

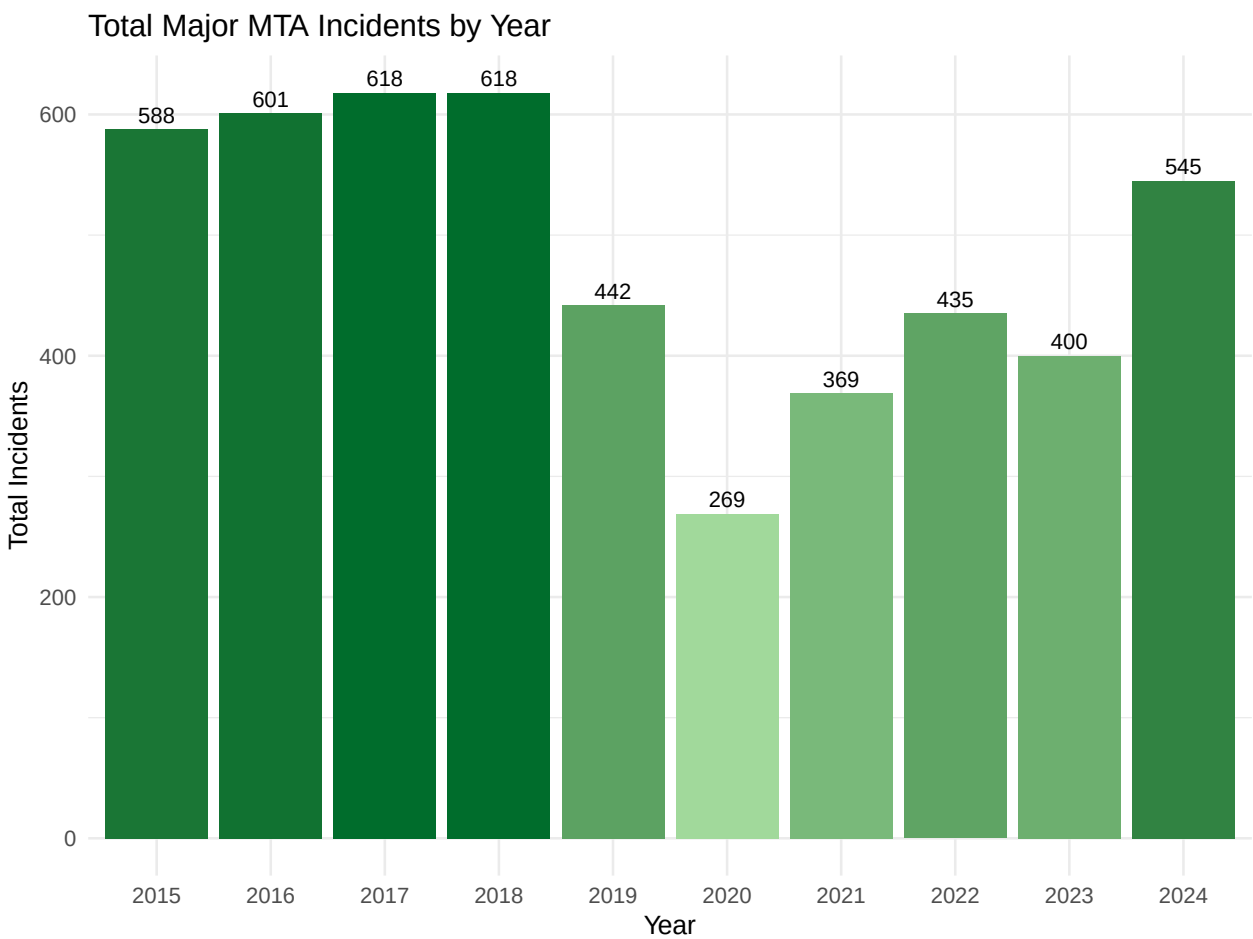
```
ggplot(category_totals, aes(x = reorder(category, total), y = total, fill = total)) +
  geom_col(show.legend = FALSE) +
  scale_fill_gradient(low = "#9ecae1", high = "#08519c") +
  coord_flip() +
  theme_minimal(base_size = 12) +
  labs(
    title = "Total Major MTA Incidents by Category (2015-2024)",
    x = "Category",
    y = "Total Incidents"
  )
```



This bar chart highlights which categories of disruptions account for the largest share of major incidents over the full period.

Total Incidents by Year

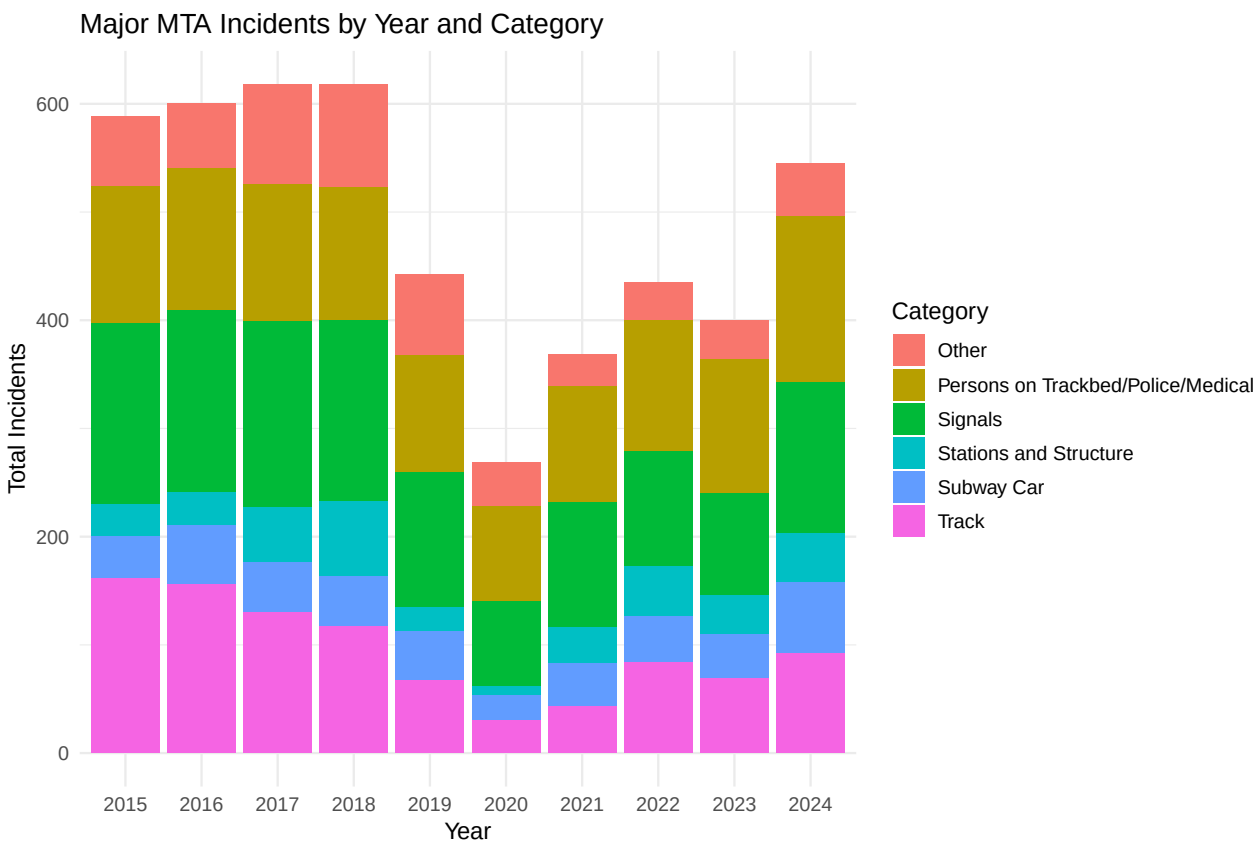
```
yearly_totals <- all_incidents %>%
  mutate(year = year(month)) %>%
  group_by(year) %>%
  summarise(total = n(), .groups = "drop")

ggplot(yearly_totals, aes(x = factor(year), y = total, fill = total)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = total), vjust = -0.4, size = 3.5) +
  scale_fill_gradient(low = "#a1d99b", high = "#006d2c") +
  theme_minimal(base_size = 12) +
  labs(
    title = "Total Major MTA Incidents by Year",
    x = "Year",
    y = "Total Incidents"
  )
```



The year-over-year bar chart makes it easy to see annual fluctuations, including the pandemic-era drop in 2020.

Stacked Bar Chart: Incidents by Year and Category

```r
yearly_category <- all_incidents %>%
  mutate(year = year(month)) %>%
  group_by(year, category) %>%
  summarise(total = n(), .groups = "drop")

ggplot(yearly_category, aes(x = factor(year), y = total, fill = category)) +
  geom_col(position = "stack") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "right") +
  labs(
    title = "Major MTA Incidents by Year and Category",
    x = "Year",
    y = "Total Incidents",
    fill = "Category"
  )
```

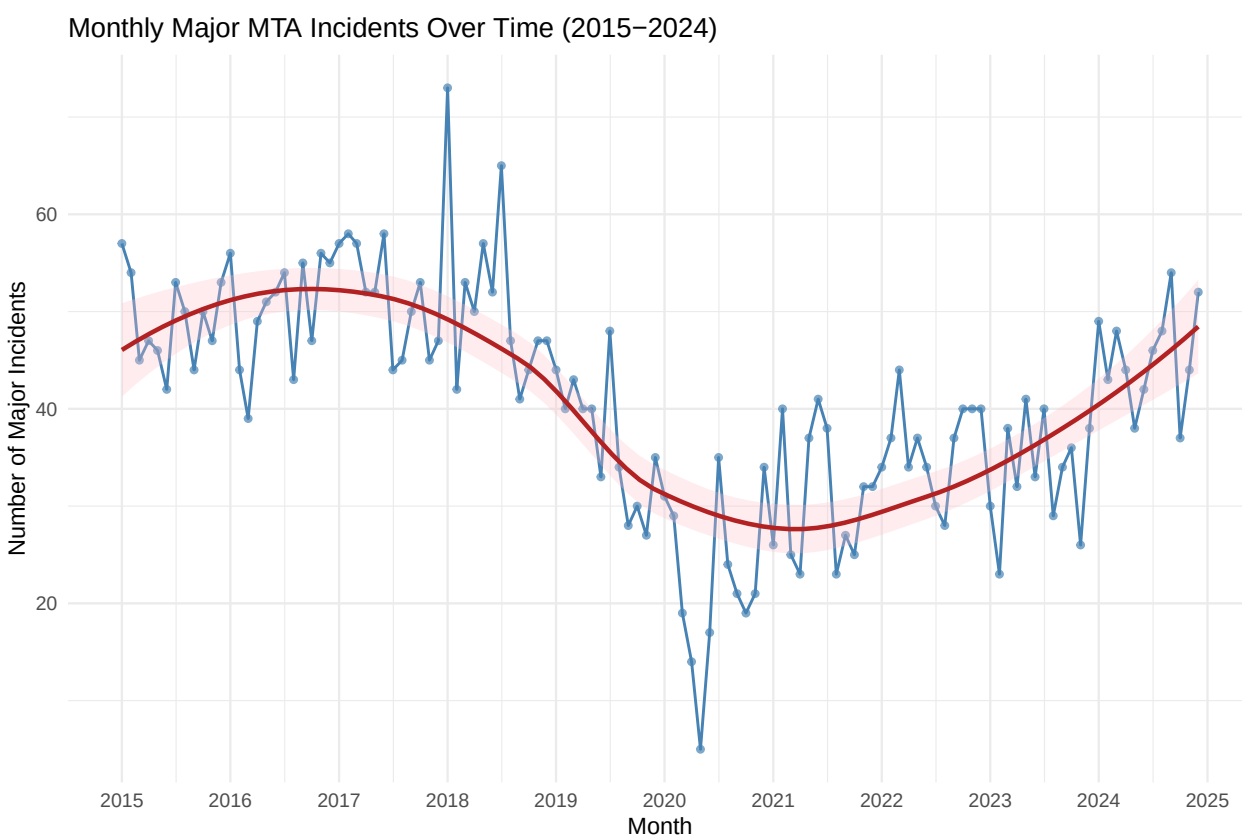


The stacked bar chart shows how the composition of incident categories shifts across years.

Time Series of Monthly Major Incidents

```
ggplot(monthly_totals, aes(x = month, y = total)) +
  geom_line(color = "steelblue", linewidth = 0.7) +
  geom_point(color = "steelblue", size = 1.4, alpha = 0.7) +
  geom_smooth(method = "loess", se = TRUE,
              color = "firebrick", fill = "pink", alpha = 0.3) +
  scale_x_date(date_breaks = "1 year", date_labels = "%Y") +
  theme_minimal(base_size = 12) +
  labs(
    title = "Monthly Major MTA Incidents Over Time (2015-2024)",
    x = "Month",
    y = "Number of Major Incidents"
  )
```

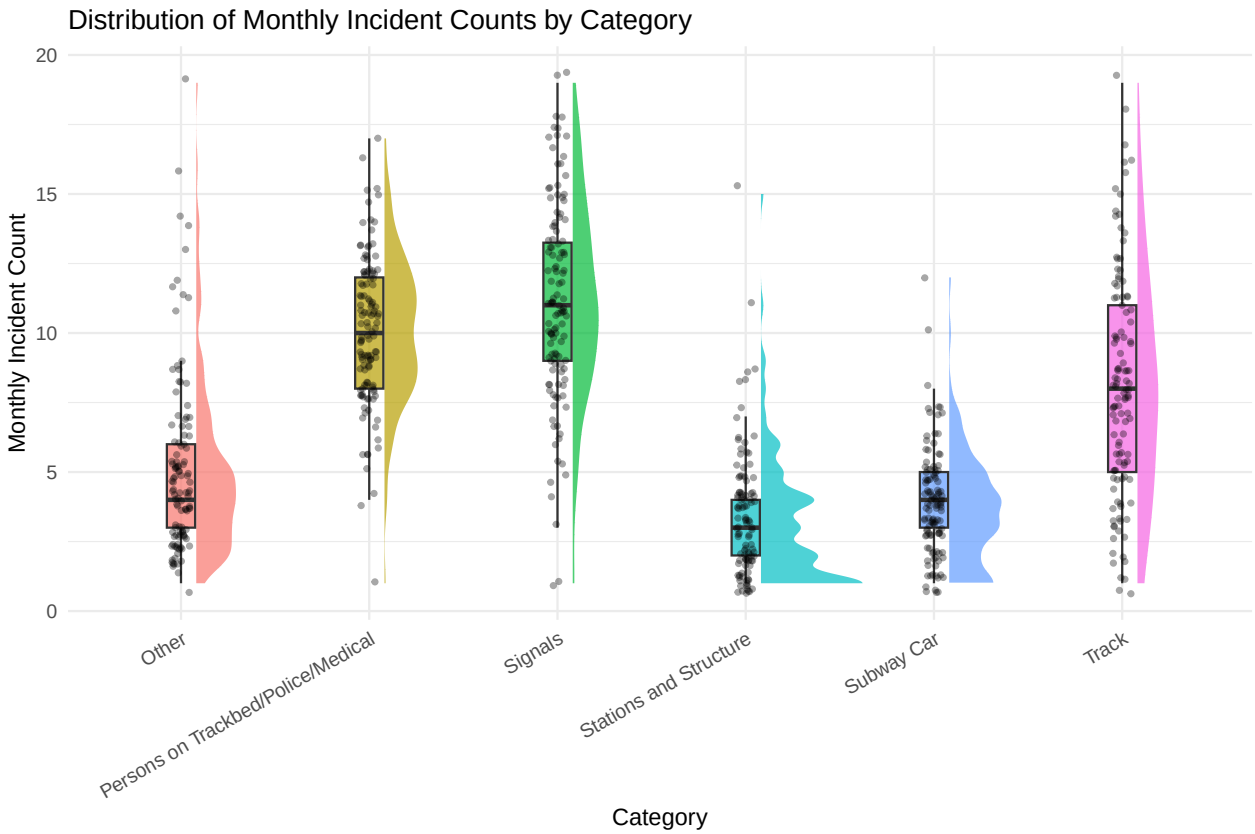
```
`geom_smooth()` using formula = 'y ~ x'
```



The time series plot shows the monthly trend across the full ten-year window, with a LOESS smoother highlighting the underlying trajectory and notable departures around the pandemic.

Half-Violin Plot: Distribution of Monthly Incidents by Category

```r
ggplot(summary_df, aes(x = category, y = count, fill = category)) +
  stat_halfeye(
    adjust = 0.7,
    width = 0.6,
    justification = -0.15,
    .width = 0,
    point_colour = NA,
    alpha = 0.7
  ) +
  geom_boxplot(
    width = 0.15,
    outlier.shape = NA,
    alpha = 0.7
  ) +
  geom_jitter(width = 0.05, alpha = 0.35, size = 1) +
  theme_minimal(base_size = 12) +
  theme(
    axis.text.x = element_text(angle = 30, hjust = 1),
    legend.position = "none"
  ) +
  labs(
    title = "Distribution of Monthly Incident Counts by Category",
    x = "Category",
    y = "Monthly Incident Count"
  )
```



The half-violin plot pairs the density of monthly incident counts (right side) with a boxplot summary (left side) for each category. This makes it easy to compare both the spread and the typical level of incidents across categories, and highlights categories that occasionally produce extreme months.

Download and Prepare Weather Data

```
base_url <- "https://www.ncei.noaa.gov/access/services/data/v1"

res <- GET(
  url = base_url,
  query = list(
    dataset = "daily-summaries",
    stations = "USW00094728",
    startDate = "2015-01-01",
    endDate = "2024-12-31",
    dataTypes = "TMAX,TMIN,PRCP,SNOW,SNWD",
    format = "json",
    units = "standard",
    includeAttributes = "false"
  ),
  timeout(60)
)
```

```r
stop_for_status(res)

weather_daily <- content(res, as = "parsed", type = "application/json") %>%
  bind_rows() %>%
  mutate(
    DATE = as.Date(DATE),
    TMAX = as.numeric(TMAX),
    TMIN = as.numeric(TMIN),
    PRCP = as.numeric(PRCP),
    SNOW = as.numeric(SNOW),
    SNWD = as.numeric(SNWD),
    month = floor_date(DATE, "month"),
    heavy_precip_day = PRCP >= 0.5,
    hot_day = TMAX >= 90
  )

weather_monthly <- weather_daily %>%
  group_by(month) %>%
  summarise(
    avg_tmax = mean(TMAX, na.rm = TRUE),
    avg_tmin = mean(TMIN, na.rm = TRUE),
    total_prcp = sum(PRCP, na.rm = TRUE),
    total_snow = sum(SNOW, na.rm = TRUE),
    heavy_precip_days = sum(heavy_precip_day, na.rm = TRUE),
    hot_days = sum(hot_day, na.rm = TRUE),
    n_days = n(),
    .groups = "drop"
  ) %>%
  arrange(month)
```

Combine Incidents and Weather Data

```r
combined <- monthly_totals %>%
  mutate(month = as.Date(month)) %>%
  left_join(weather_monthly, by = "month")
```

Correlation Analysis

```r
numeric_vars <- combined %>%
  select(
    total, avg_tmax, avg_tmin, total_prcp, total_snow,
```

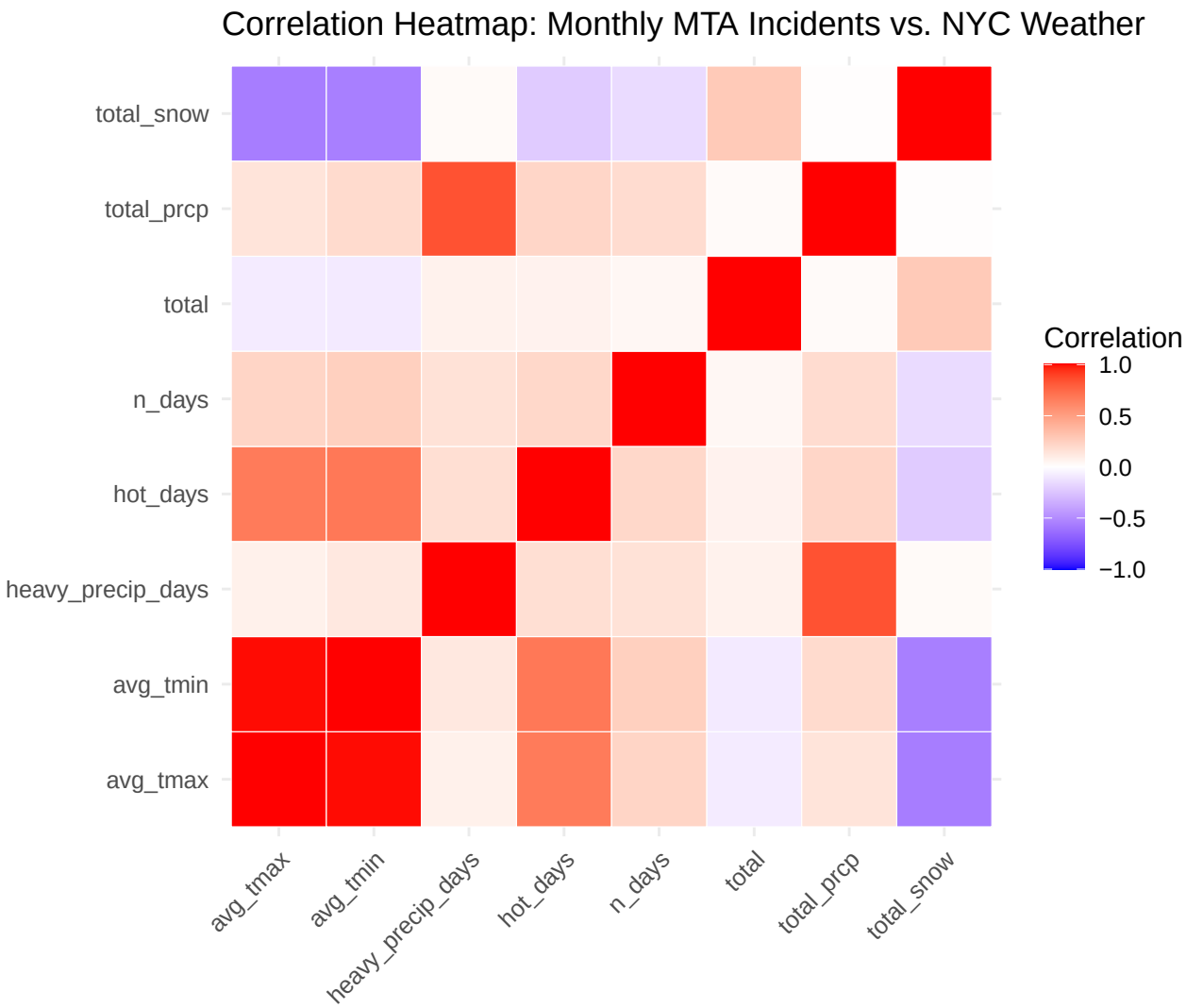
```
    heavy_precip_days, hot_days, n_days
  )

cor_mat <- cor(numeric_vars, use = "pairwise.complete.obs")
cor_mat
```

```
                       total    avg_tmax    avg_tmin  total_prcp   total_snow
total              1.00000000 -0.08513800 -0.08701713 0.024092758  0.277298731
avg_tmax          -0.08513800  1.00000000  0.99210678 0.141711025 -0.561485258
avg_tmin          -0.08701713  0.99210678  1.00000000 0.190049044 -0.555586515
total_prcp         0.02409276  0.14171102  0.19004904 1.000000000  0.009112456
total_snow         0.27729873 -0.56148526 -0.55558652 0.009112456  1.000000000
heavy_precip_days  0.07065120  0.07532838  0.11880931 0.832853773  0.028099085
hot_days           0.06611572  0.66671750  0.67727388 0.212123798 -0.221330811
n_days             0.04373678  0.21932250  0.24820714 0.182489198 -0.154633219
                  heavy_precip_days    hot_days      n_days
total                    0.07065120  0.06611572  0.04373678
avg_tmax                 0.07532838  0.66671750  0.21932250
avg_tmin                 0.11880931  0.67727388  0.24820714
total_prcp               0.83285377  0.21212380  0.18248920
total_snow               0.02809908 -0.22133081 -0.15463322
heavy_precip_days        1.00000000  0.16796088  0.15381290
hot_days                 0.16796088  1.00000000  0.20668057
n_days                   0.15381290  0.20668057  1.00000000
```

The correlation matrix shows that snowfall has the strongest relationship with incidents among the weather variables, while most other weather measures have weak associations.

```
cor_long <- as.data.frame(cor_mat) %>%
  mutate(var1 = rownames(.)) %>%
  pivot_longer(
    cols = -var1,
    names_to = "var2",
    values_to = "correlation"
  )

ggplot(cor_long, aes(x = var1, y = var2, fill = correlation)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = 0,
    limits = c(-1, 1)
  ) +
  coord_fixed() +
```

```r
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(
    x = NULL,
    y = NULL,
    fill = "Correlation",
    title = "Correlation Heatmap: Monthly MTA Incidents vs. NYC Weather"
  )
```



The heatmap confirms that snowfall is the weather feature most visibly associated with incident totals.

Initial Regression Models Using Weather Only

```r
model1 <- lm(total ~ avg_tmax + total_prcp + total_snow, data = combined)
summary(model1)
```

```

Call:
lm(formula = total ~ avg_tmax + total_prcp + total_snow, data = combined)

Residuals:
    Min      1Q  Median      3Q     Max 
-34.428  -7.149   0.903   7.816  27.177 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept) 34.25295    5.47981   6.251 7.01e-09 ***
avg_tmax     0.07399    0.07944   0.931  0.35358    
total_prcp   0.03394    0.46113   0.074  0.94146    
total_snow   0.77118    0.24917   3.095  0.00247 ** 
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 11.18 on 116 degrees of freedom
Multiple R-squared:  0.08421,	Adjusted R-squared:  0.06052 
F-statistic: 3.555 on 3 and 116 DF,  p-value: 0.01657
```

```r
model2 <- lm(total ~ hot_days + heavy_precip_days + total_snow, data = combined)
summary(model2)
```

```

Call:
lm(formula = total ~ hot_days + heavy_precip_days + total_snow, 
    data = combined)

Residuals:
    Min      1Q  Median      3Q     Max 
-32.982  -7.625   1.058   8.012  26.894 

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)    
(Intercept)        37.7176     2.0129  18.738  < 2e-16 ***
hot_days            0.5010     0.3643   1.375  0.17164    
heavy_precip_days   0.2647     0.5817   0.455  0.65000    
total_snow          0.7017     0.2094   3.352  0.00109 ** 
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 11.11 on 116 degrees of freedom
Multiple R-squared:  0.0956,    Adjusted R-squared:  0.07221
F-statistic: 4.087 on 3 and 116 DF,  p-value: 0.008463
```

These weather-only models show that snowfall is significant, but the overall explanatory power is low.

```r
vars_for_cor <- combined %>%
  select(avg_tmax, avg_tmin, total_prcp, total_snow, heavy_precip_days, hot_days)

cor(vars_for_cor, use = "pairwise.complete.obs")
```

```
                     avg_tmax   avg_tmin  total_prcp   total_snow
avg_tmax           1.00000000  0.9921068 0.141711025 -0.561485258
avg_tmin           0.99210678  1.0000000 0.190049044 -0.555586515
total_prcp         0.14171102  0.1900490 1.000000000  0.009112456
total_snow        -0.56148526 -0.5555865 0.009112456  1.000000000
heavy_precip_days  0.07532838  0.1188093 0.832853773  0.028099085
hot_days           0.66671750  0.6772739 0.212123798 -0.221330811
                  heavy_precip_days   hot_days
avg_tmax                 0.07532838  0.6667175
avg_tmin                 0.11880931  0.6772739
total_prcp               0.83285377  0.2121238
total_snow               0.02809908 -0.2213308
heavy_precip_days        1.00000000  0.1679609
hot_days                 0.16796088  1.0000000
```

```r
vif(model1)
```

```
  avg_tmax total_prcp total_snow
  1.507899   1.032596   1.477741
```

```r
vif(model2)
```

```
         hot_days heavy_precip_days        total_snow
         1.086189          1.033796          1.056381
```

The low VIF values indicate that multicollinearity is not a major concern in these models.

Simple Linear Regression with Snow Only

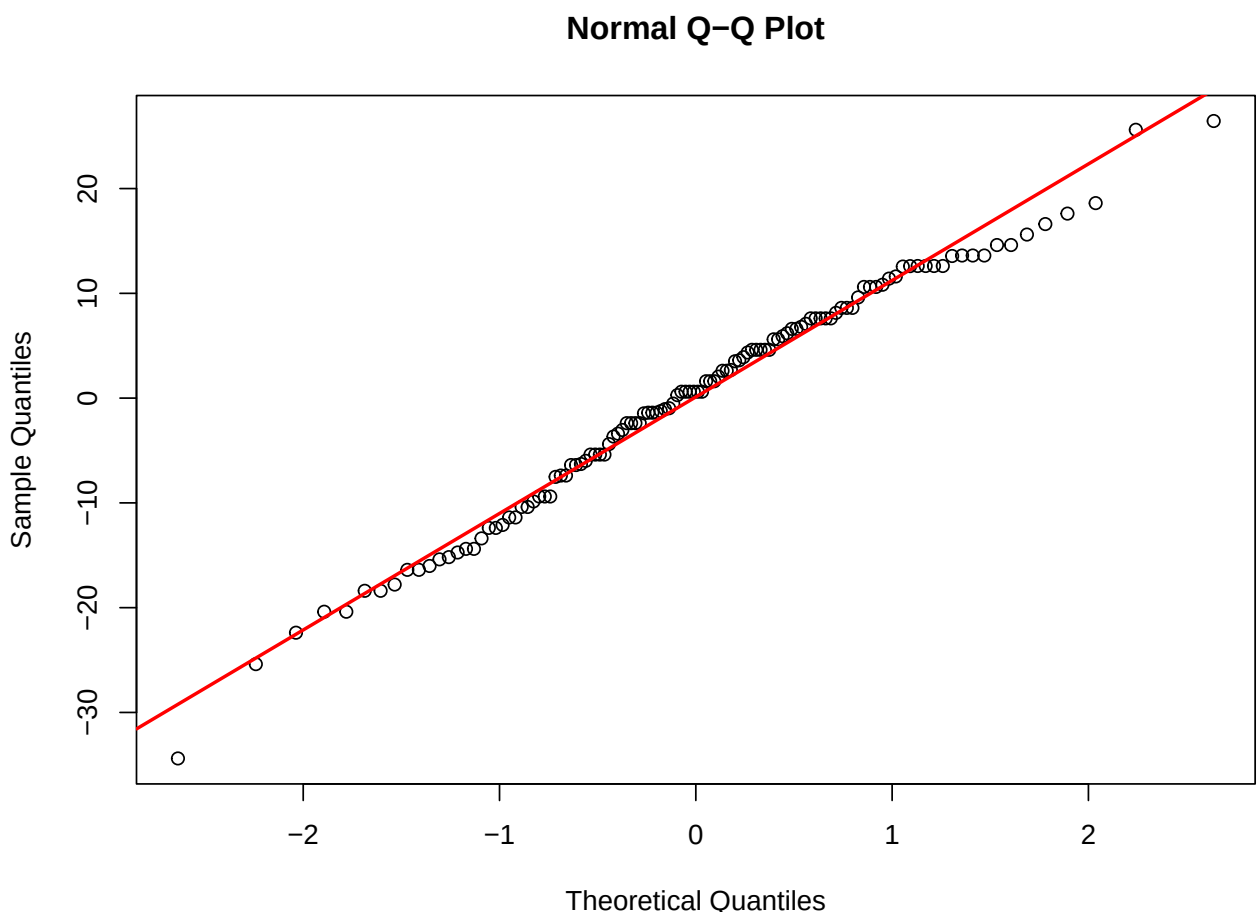
```r
snow_model <- lm(total ~ total_snow, data = combined)
summary(snow_model)
```

```

Call:
lm(formula = total ~ total_snow, data = combined)

Residuals:
   Min     1Q Median     3Q    Max 
-34.39  -7.39   0.61   7.61  26.45 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  39.3895     1.0992  35.833  < 2e-16 ***
total_snow    0.6397     0.2040   3.135  0.00217 ** 
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 11.12 on 118 degrees of freedom
Multiple R-squared:  0.07689,	Adjusted R-squared:  0.06907 
F-statistic: 9.829 on 1 and 118 DF,  p-value: 0.002168
```

The snow-only model shows that snowfall is a significant predictor, but it explains only a small portion of the variation in incidents.

Q-Q Plot for Snow-Only Model

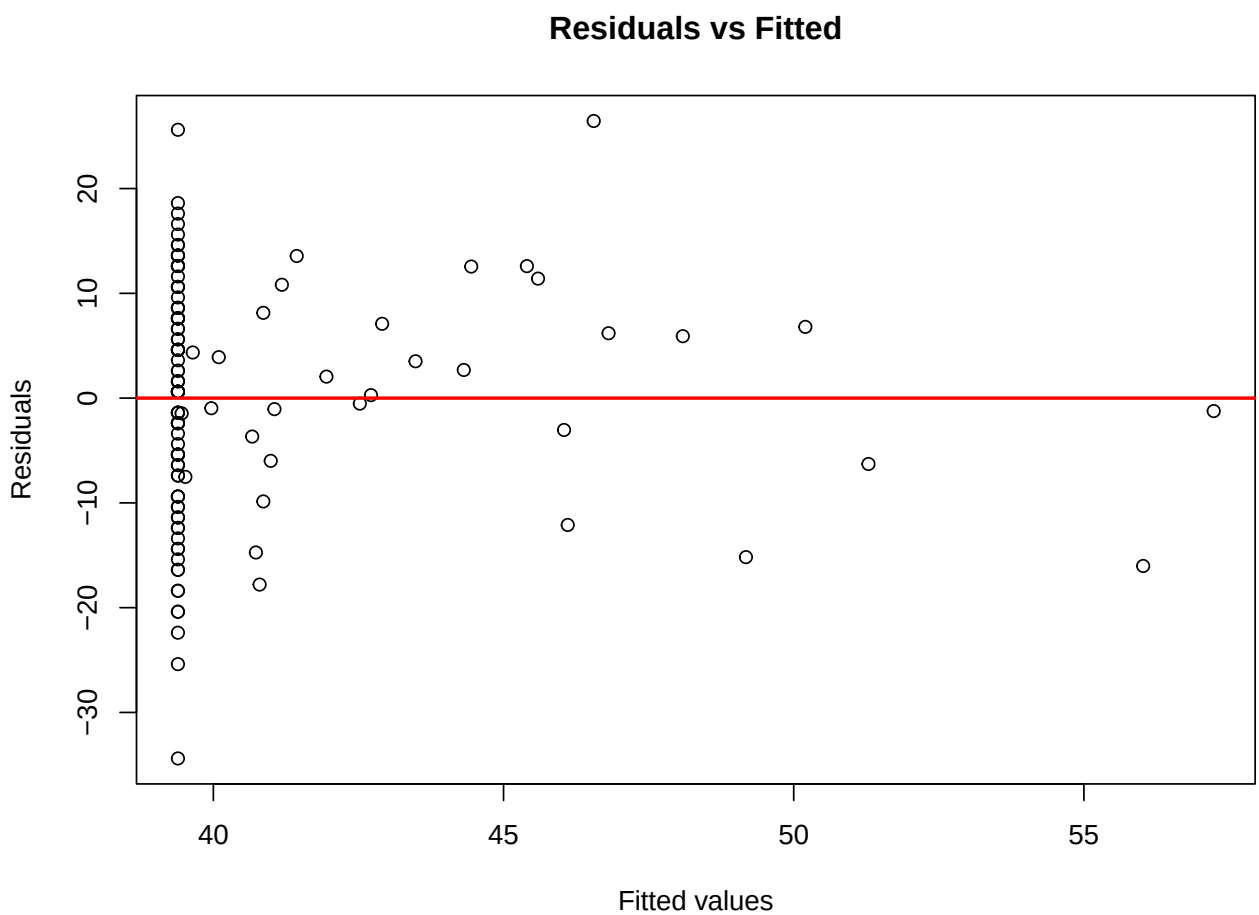
```r
qqnorm(residuals(snow_model))
qqline(residuals(snow_model), col = "red", lwd = 2)
```



The residuals are approximately normal, with some deviation in the tails.

Residuals vs Fitted for Snow-Only Model

```
plot(
  fitted(snow_model),
  residuals(snow_model),
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)
abline(h = 0, col = "red", lwd = 2)
```



The residual pattern suggests the snow-only model misses important variation in the data.

Log-Transformed Snow-Only Model

```
log_snow_model <- lm(log(total) ~ total_snow, data = combined)
summary(log_snow_model)
```

```
Call:
lm(formula = log(total) ~ total_snow, data = combined)

Residuals:
     Min       1Q   Median       3Q      Max 
-2.01088 -0.15458  0.06856  0.22983  0.55407 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept) 3.620319   0.034280 105.609  < 2e-16 ***
total_snow  0.016792   0.006363   2.639  0.00944 ** 
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3469 on 118 degrees of freedom
Multiple R-squared:  0.05573,	Adjusted R-squared:  0.04773
F-statistic: 6.964 on 1 and 118 DF,  p-value: 0.009439
```

The log transformation changes the relationship slightly, but the model still leaves substantial variation unexplained because it does not yet account for differences in subway usage.

Add Subway Ridership Data (2020 and Later)

```r
ridership <- read_csv("~/Downloads/MTA RIDERSHIP SINCE 2020.csv", show_col_types = FALSE)

ridership_monthly <- ridership %>%
  filter(Mode == "Subway") %>%
  mutate(
    Date = mdy(Date),
    month = floor_date(Date, "month")
  ) %>%
  group_by(month) %>%
  summarise(
    ridership = sum(Count, na.rm = TRUE),
    .groups = "drop"
  )
```

Ridership is aggregated to the monthly level so it matches the monthly incident data.

```r
combined_2020 <- combined %>%
  mutate(month = as.Date(month)) %>%
  filter(month >= as.Date("2020-01-01")) %>%
  left_join(ridership_monthly, by = "month") %>%
  filter(!is.na(ridership)) %>%
  mutate(ridership_millions = ridership / 1e6)
```

Restricting to 2020 onward allows incidents, weather, and ridership to be analyzed on the same time scale.

Linear Model with Snow and Ridership

```r
final_model <- lm(total ~ total_snow + ridership_millions, data = combined_2020)
summary(final_model)
```

```
Call:
lm(formula = total ~ total_snow + ridership_millions, data = combined_2020)

Residuals:
     Min       1Q   Median       3Q      Max 
-13.4890  -4.8526  -0.2617   3.5407  15.1456 

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)    
(Intercept)        10.38257    3.05440   3.399  0.00126 ** 
total_snow          0.70220    0.22300   3.149  0.00265 ** 
ridership_millions  0.29136    0.03677   7.923 1.16e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.815 on 55 degrees of freedom
Multiple R-squared:  0.5393,	Adjusted R-squared:  0.5226 
F-statistic:  32.2 on 2 and 55 DF,  p-value: 5.531e-10
```

The linear model with snow and ridership substantially improves on the earlier weather-only models. Both snowfall and ridership are statistically significant predictors of monthly major incidents.

An increase of 1 million riders is associated with additional major incidents, holding snowfall constant. Likewise, each additional inch of snow is associated with more incidents, holding ridership constant.

Incident Rate Model

```r
combined_2020 <- combined_2020 %>%
  mutate(incident_rate = total / ridership)

rate_model <- lm(incident_rate ~ total_snow, data = combined_2020)
summary(rate_model)
```

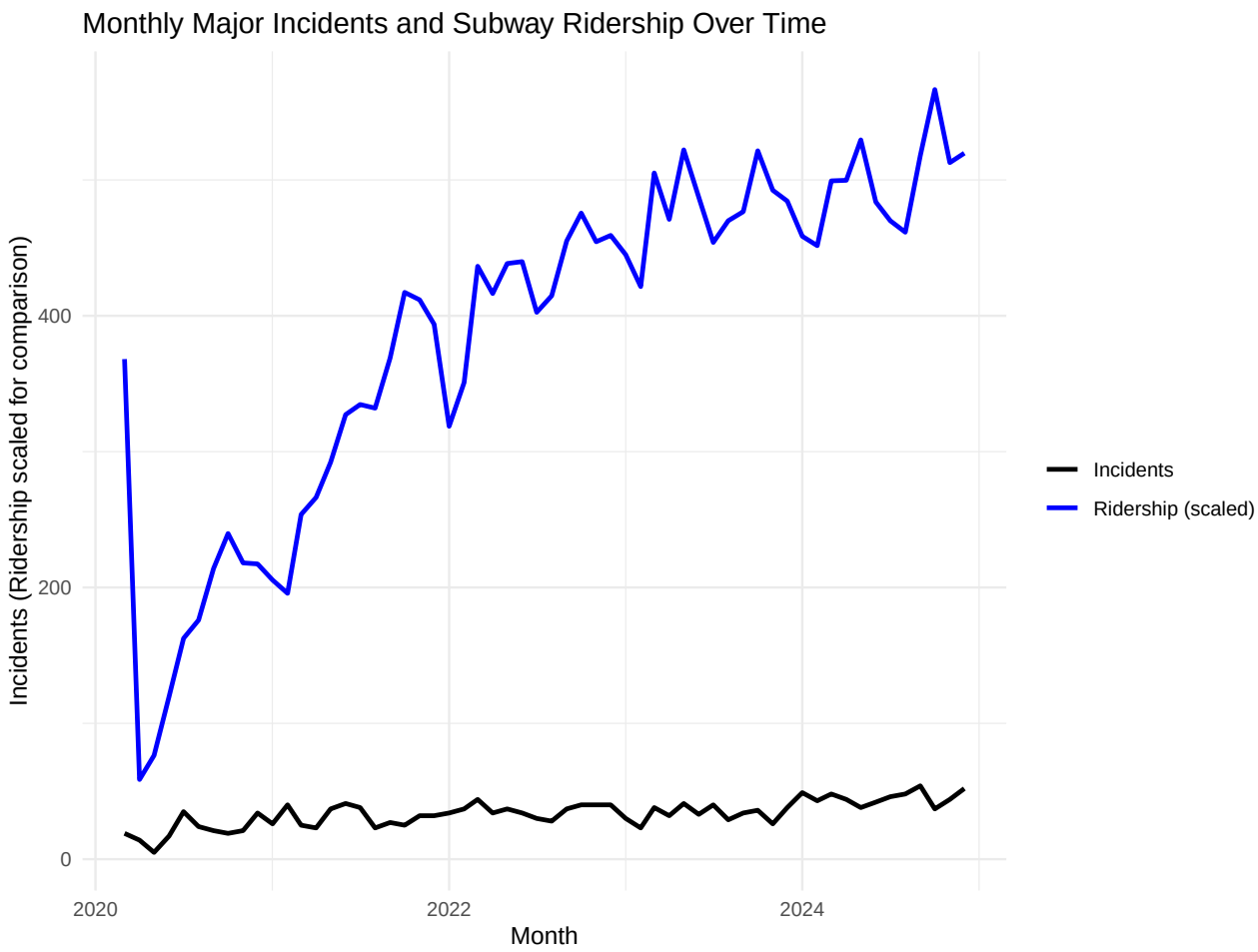
```
Call:
lm(formula = incident_rate ~ total_snow, data = combined_2020)

Residuals:
       Min         1Q     Median         3Q        Max 
-2.199e-07 -9.424e-08 -3.022e-08  3.959e-08  7.420e-07 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept) 4.514e-07  2.300e-08  19.629  < 2e-16 ***
total_snow  1.877e-08  5.345e-09   3.512 0.000887 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.683e-07 on 56 degrees of freedom
Multiple R-squared:  0.1805,    Adjusted R-squared:  0.1659
F-statistic: 12.33 on 1 and 56 DF,  p-value: 0.0008871
```

The incident rate model shows that snowfall remains significant even after adjusting for ridership.

Time Series of Incidents and Ridership

```r
ggplot(combined_2020, aes(x = month)) +
  geom_line(aes(y = total, color = "Incidents"), linewidth = 1) +
  geom_line(aes(y = ridership_millions * 5, color = "Ridership (scaled)"), linewidth = 1) +
  scale_color_manual(values = c("Incidents" = "black", "Ridership (scaled)" = "blue")) +
  theme_minimal() +
  labs(
    title = "Monthly Major Incidents and Subway Ridership Over Time",
    x = "Month",
    y = "Incidents (Ridership scaled for comparison)",
    color = ""
  )
```

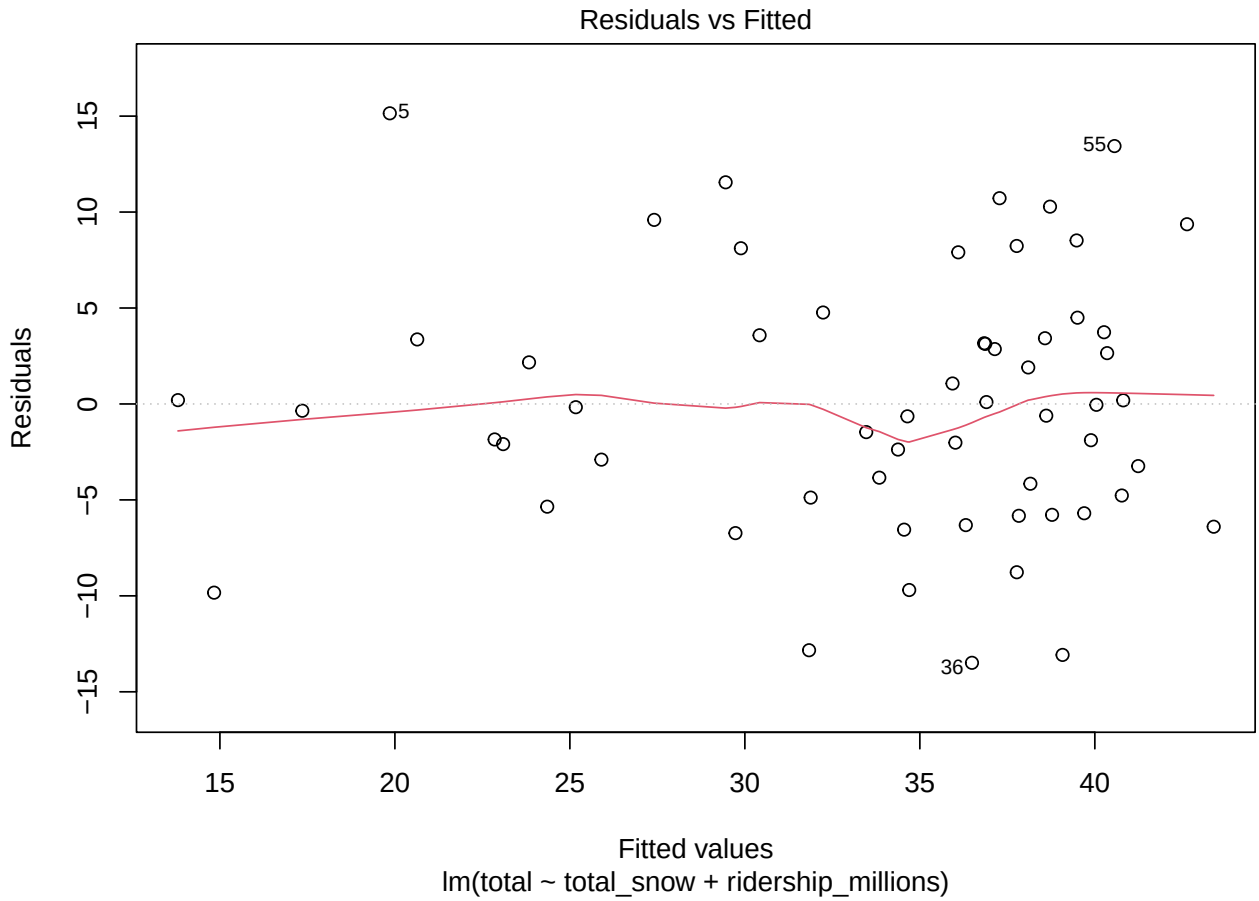


Incidents and ridership move together over time, suggesting that higher system usage is associated with more disruptions.

Diagnostics for Linear Model

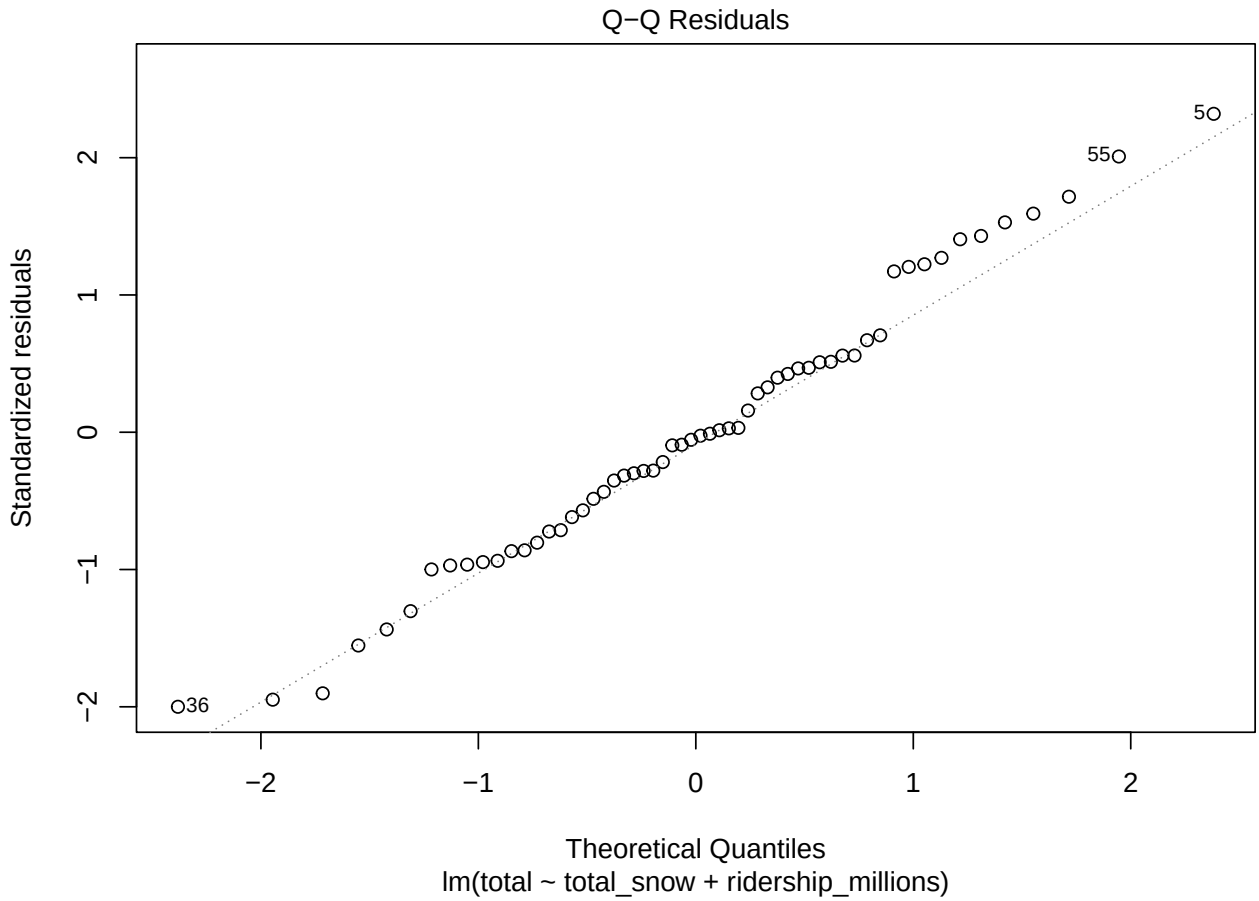
Residuals vs Fitted

```
plot(final_model, which = 1)
```



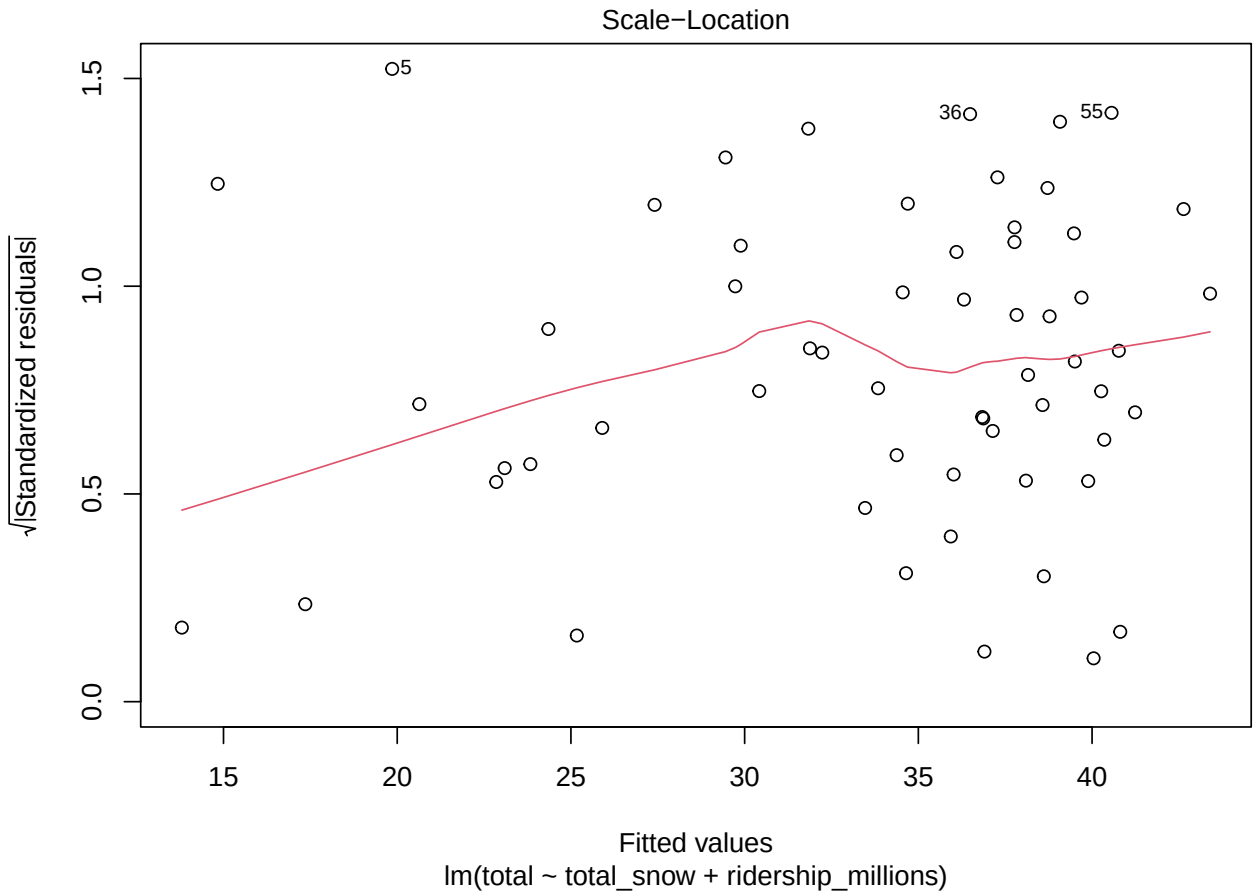
Normal Q-Q

```
plot(final_model, which = 2)
```



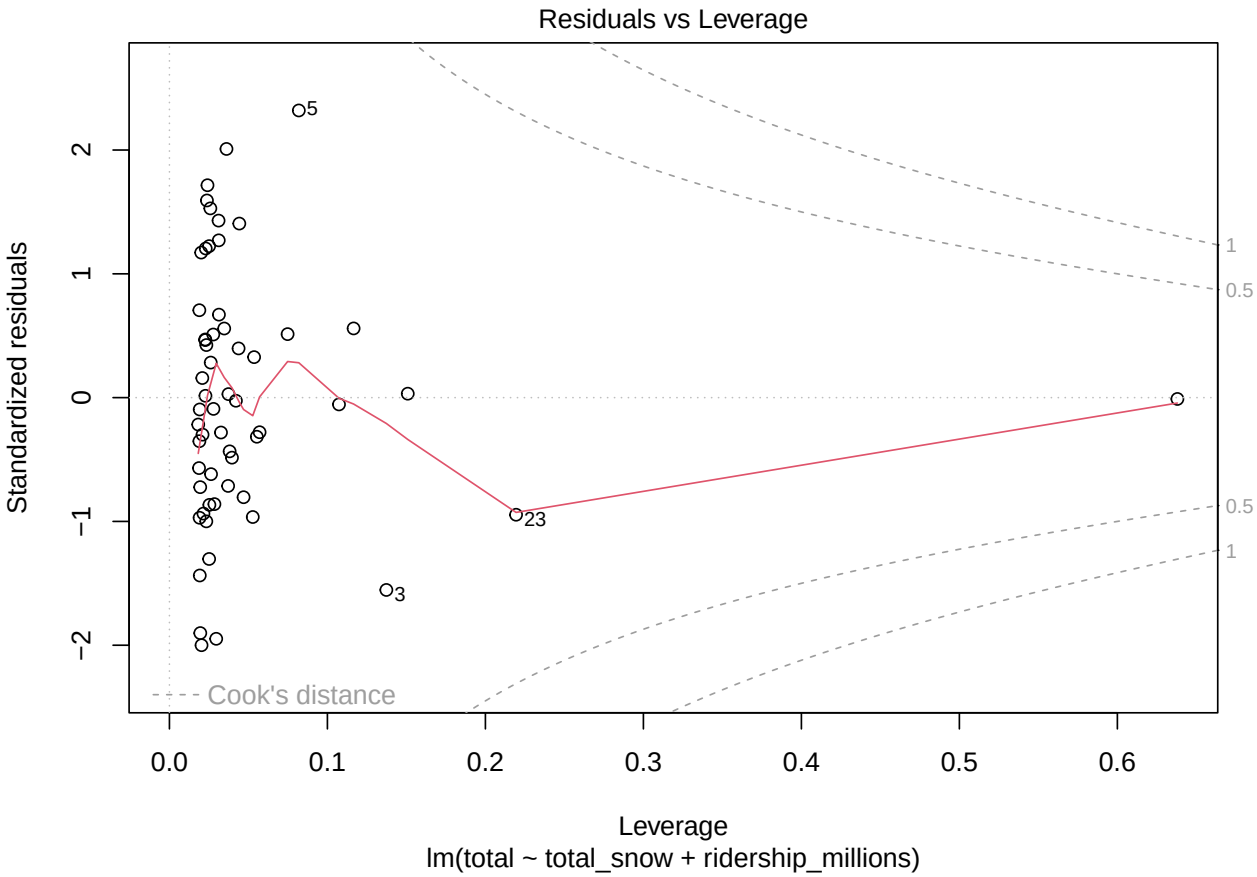
Scale-Location

```
plot(final_model, which = 3)
```



Residuals vs Leverage

```
plot(final_model, which = 5)
```




```
vif(final_model)
```

```
total_snow ridership_millions
  1.061884           1.061884
```

The linear model performs better than the earlier models, but there is still some evidence that the relationship may not be perfectly linear.

Model Comparison with Logarithmic Transformations

To further improve the model, logarithmic transformations were considered. A log-transformed response can reduce skewness and stabilize variance, while a log-log model can capture multiplicative relationships and allow coefficients to be interpreted as elasticities.

```
model_linear <- lm(total ~ total_snow + ridership_millions, data = combined_2020)

model_log <- lm(log(total) ~ total_snow + ridership_millions, data = combined_2020)

model_loglog <- lm(log(total) ~ log(total_snow + 1) + log(ridership_millions), data = combined_

AIC(model_linear, model_log, model_loglog)
```

```
             df        AIC
model_linear  4 392.128875
model_log     4  12.524819
model_loglog  4   5.402917
```

```r
summary(model_loglog)
```

```
Call:
lm(formula = log(total) ~ log(total_snow + 1) + log(ridership_millions),
    data = combined_2020)

Residuals:
     Min       1Q   Median       3Q      Max
-0.84923 -0.10489  0.01494  0.13024  0.62359

Coefficients:
                        Estimate Std. Error t value Pr(>|t|)
(Intercept)              0.75090    0.29657   2.532  0.01423 *
log(total_snow + 1)      0.13496    0.04457   3.028  0.00374 **
log(ridership_millions)  0.62641    0.06867   9.122 1.35e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.243 on 55 degrees of freedom
Multiple R-squared:  0.6117,    Adjusted R-squared:  0.5976
F-statistic: 43.33 on 2 and 55 DF,  p-value: 5.026e-12
```

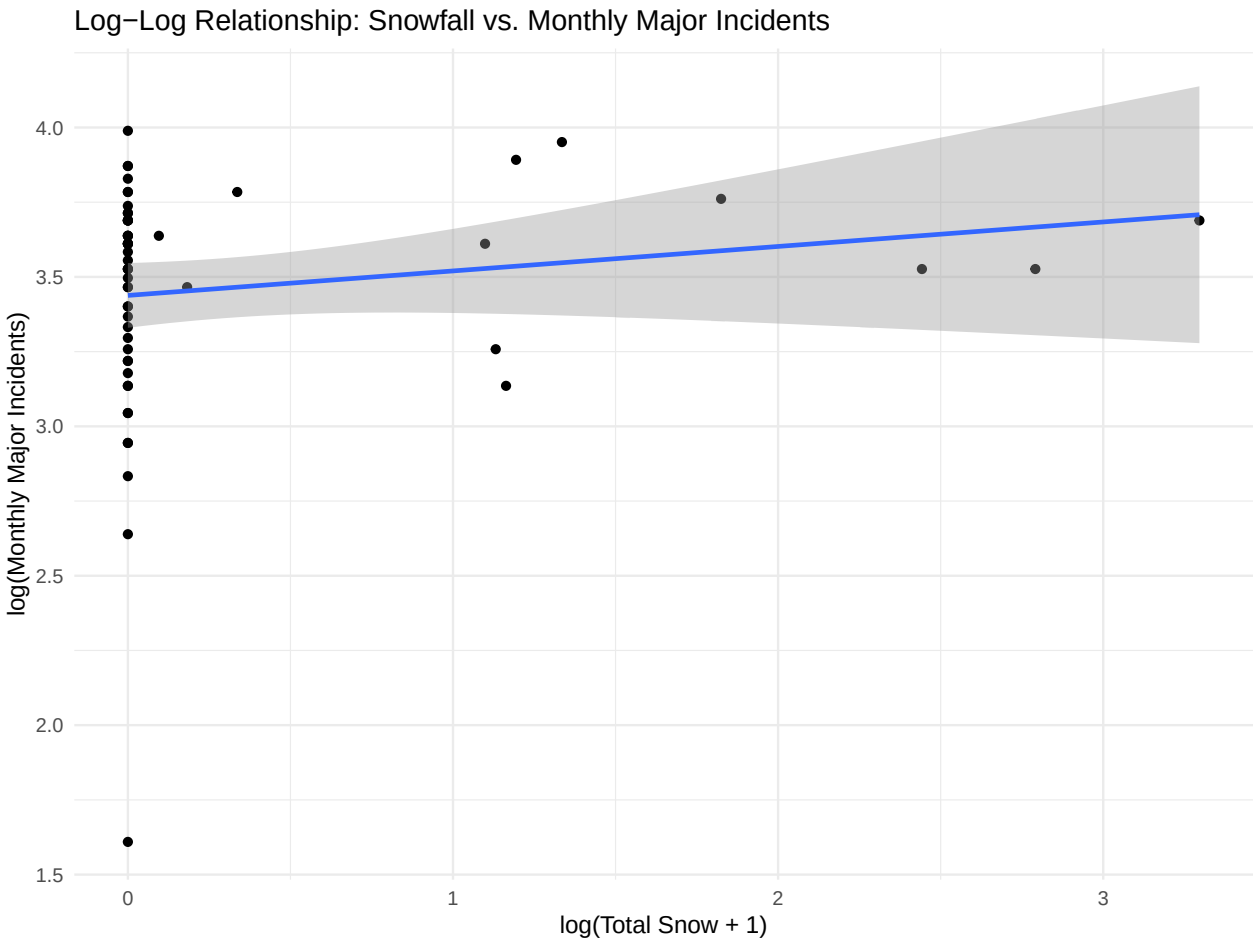
The AIC comparison shows that the log-log model provides the best fit among the three specifications. This suggests that the relationship between incidents, snowfall, and ridership is better represented on a multiplicative scale than on a purely linear one.

Additional Visualizations for the Log-Log Model

Log-Log Relationship: Snowfall vs Incidents

```r
ggplot(combined_2020, aes(x = log(total_snow + 1), y = log(total))) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE) +
  theme_minimal() +
  labs(
    title = "Log-Log Relationship: Snowfall vs. Monthly Major Incidents",
    x = "log(Total Snow + 1)",
    y = "log(Monthly Major Incidents)"
  )
```

```
`geom_smooth()` using formula = 'y ~ x'
```

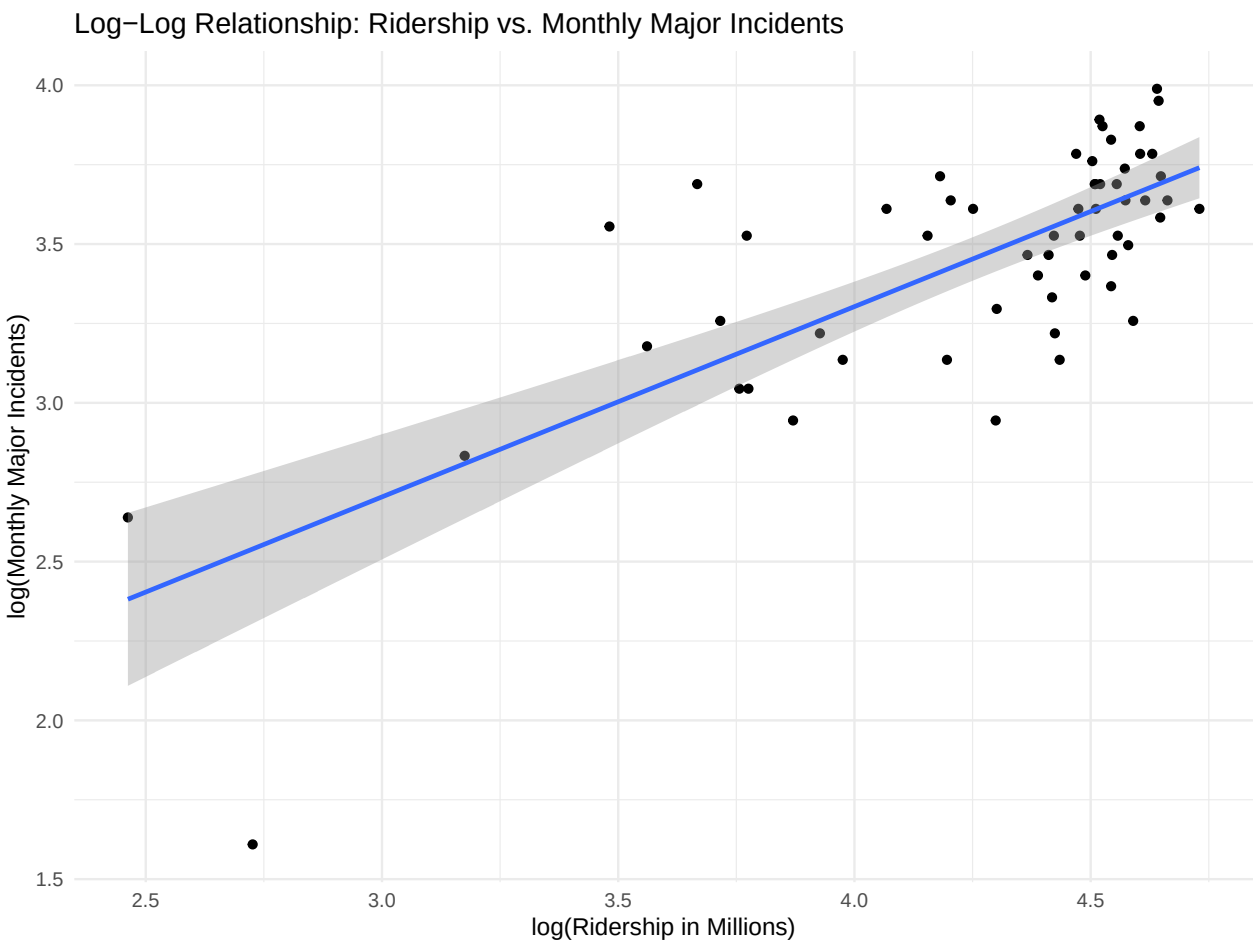


This plot shows the transformed relationship between snowfall and incidents. On the log scale, the pattern appears more linear than in the untransformed model.

Log-Log Relationship: Ridership vs Incidents

```
ggplot(combined_2020, aes(x = log(ridership_millions), y = log(total))) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE) +
  theme_minimal() +
  labs(
    title = "Log-Log Relationship: Ridership vs. Monthly Major Incidents",
    x = "log(Ridership in Millions)",
    y = "log(Monthly Major Incidents)"
  )
```

```
`geom_smooth()` using formula = 'y ~ x'
```

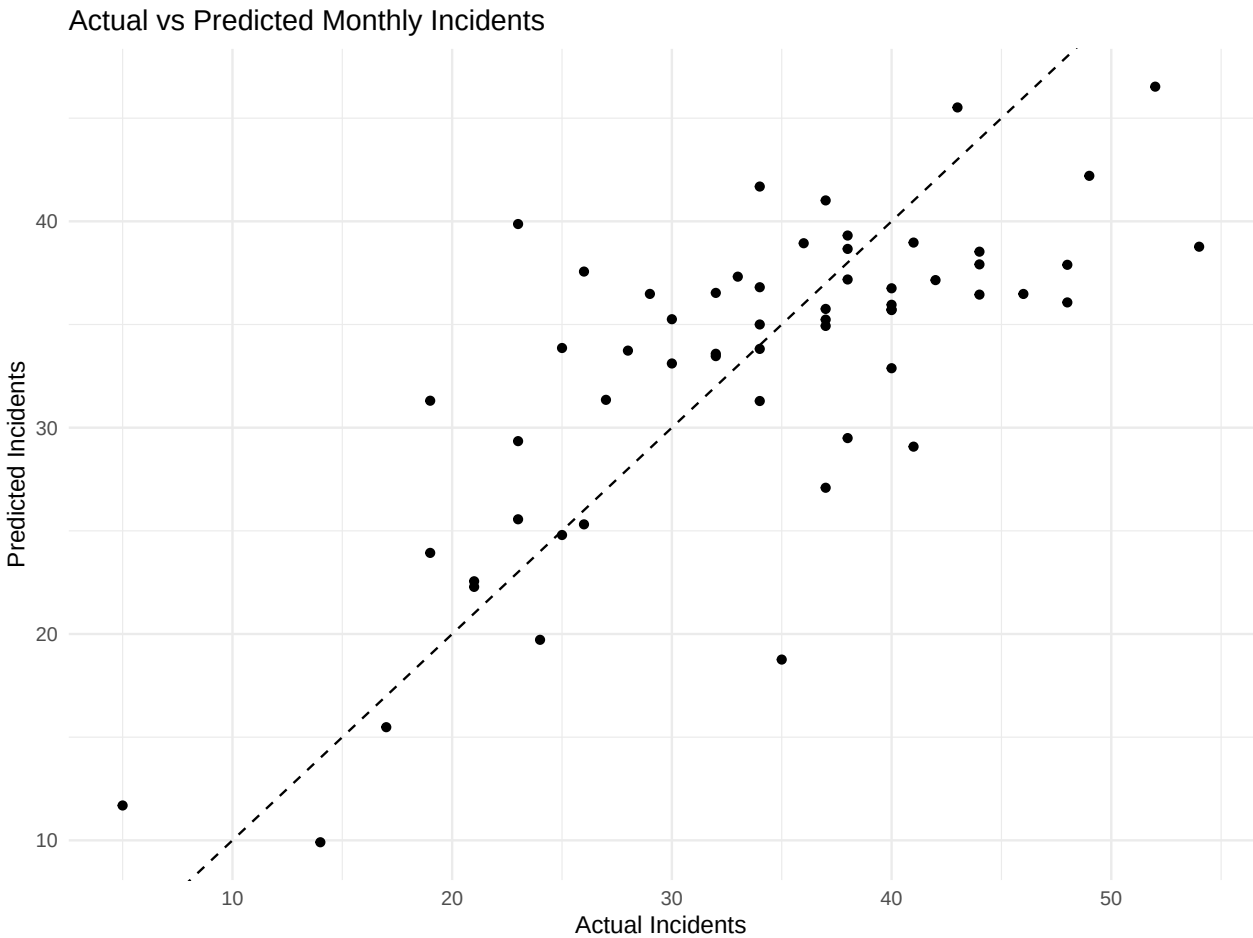


This plot shows that ridership also has a clearer linear relationship with incidents after the logarithmic transformation.

Actual vs Predicted Monthly Incidents

```
combined_2020 <- combined_2020 %>%
  mutate(
    predicted_loglog = predict(model_loglog),
    predicted_total = exp(predicted_loglog)
  )
```

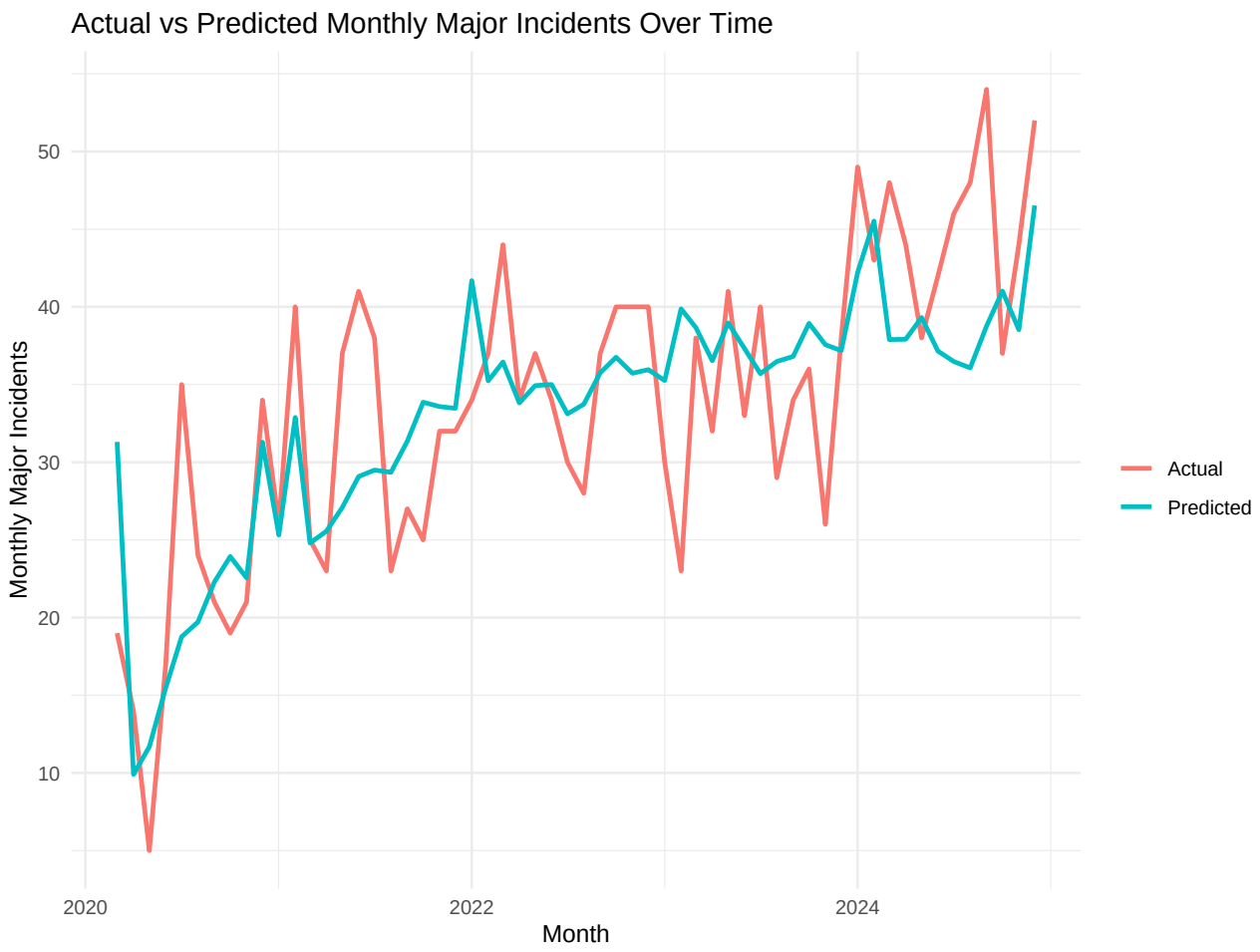
```
ggplot(combined_2020, aes(x = total, y = predicted_total)) +
  geom_point() +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  theme_minimal() +
  labs(
    title = "Actual vs Predicted Monthly Incidents",
    x = "Actual Incidents",
    y = "Predicted Incidents"
  )
```



If the model fit perfectly, the points would lie on the dashed line. The plot shows that the log-log model tracks the observed data reasonably well.

Actual vs Predicted Over Time

```
ggplot(combined_2020, aes(x = month)) +
  geom_line(aes(y = total, color = "Actual"), linewidth = 1) +
  geom_line(aes(y = predicted_total, color = "Predicted"), linewidth = 1) +
  theme_minimal() +
  labs(
    title = "Actual vs Predicted Monthly Major Incidents Over Time",
    x = "Month",
    y = "Monthly Major Incidents",
    color = ""
  )
```

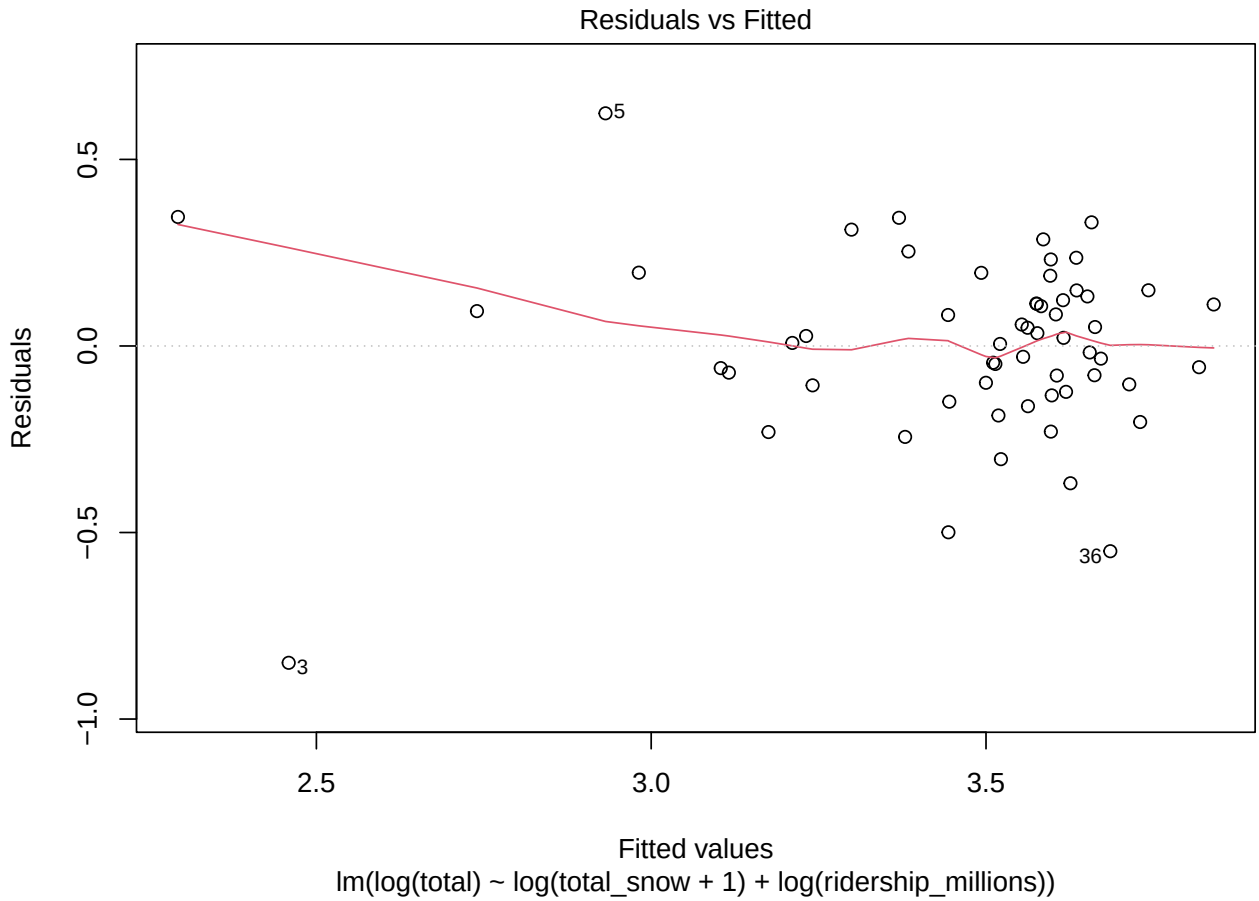


This time series comparison shows how closely the fitted values from the log-log model follow the observed monthly incident totals.

Diagnostics for the Log-Log Model

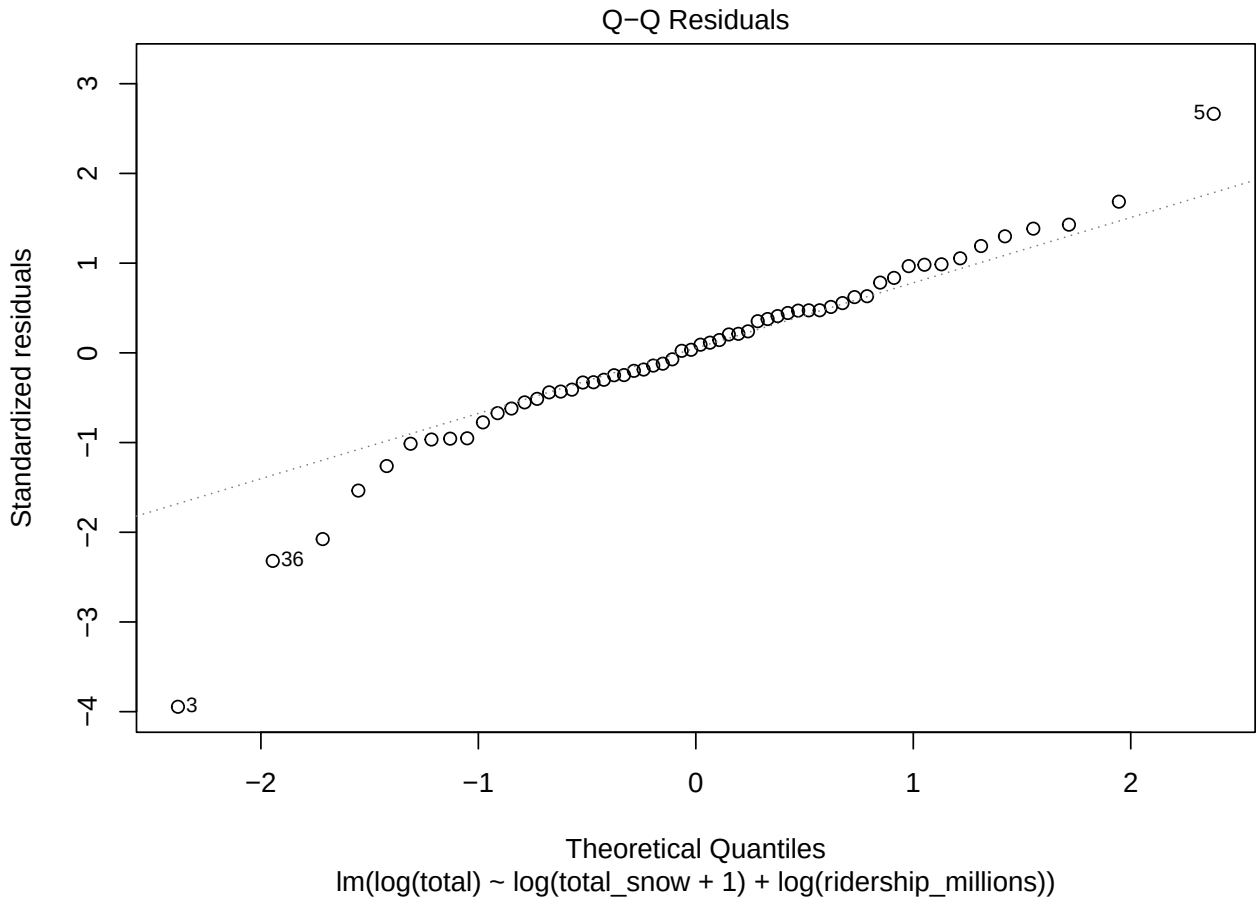
Residuals vs Fitted

```
plot(model_loglog, which = 1)
```



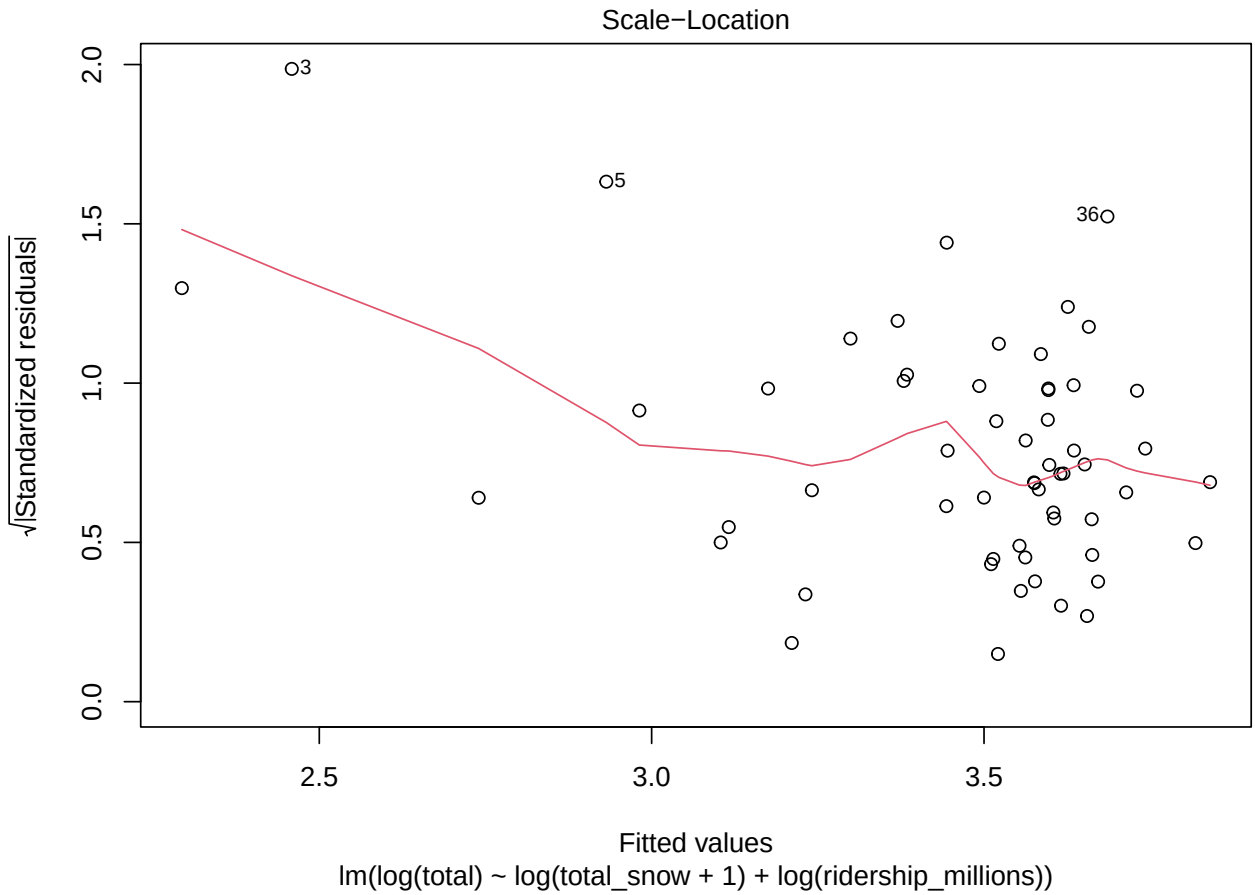
Normal Q-Q

```
plot(model_loglog, which = 2)
```



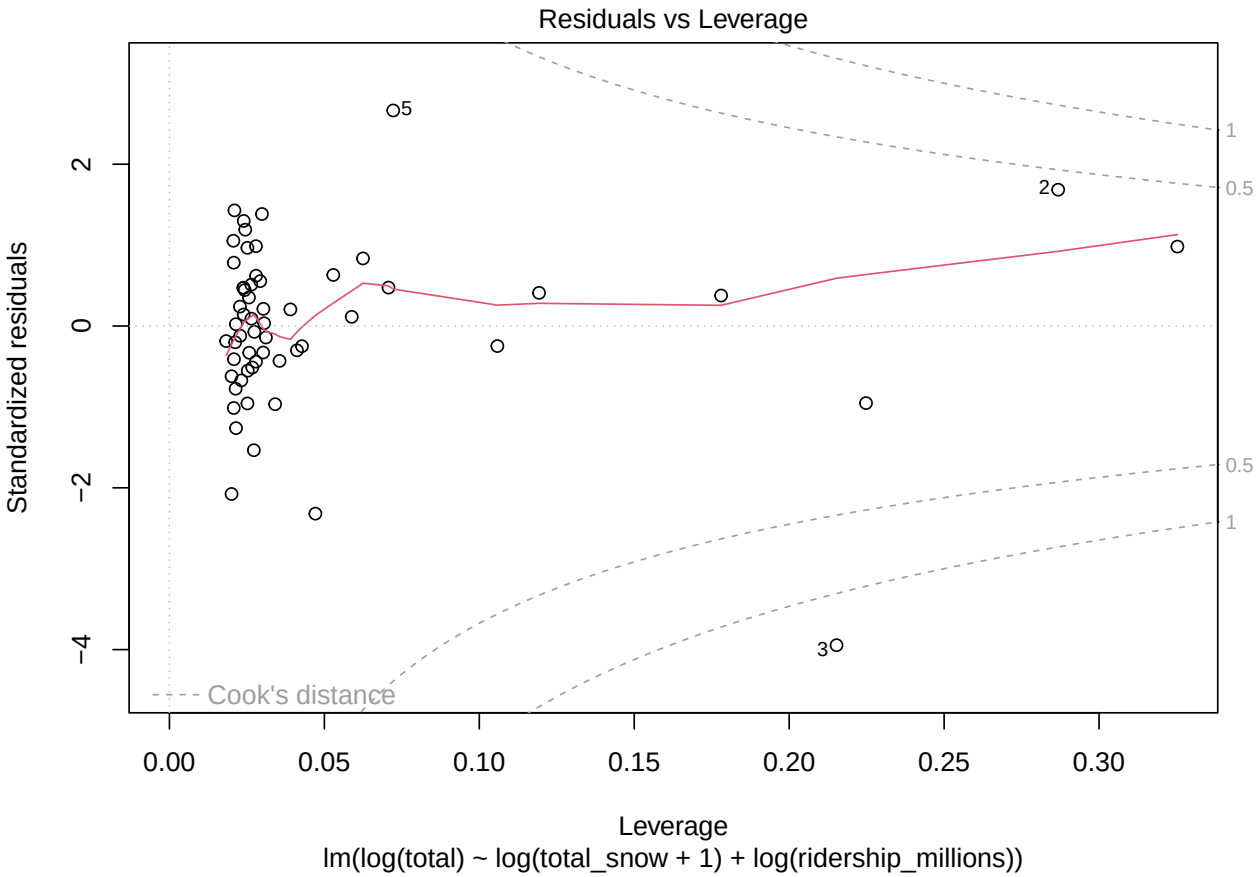
Scale-Location

```
plot(model_loglog, which = 3)
```



Residuals vs Leverage

```
plot(model_loglog, which = 5)
```



These diagnostics can be compared to the earlier linear-model diagnostics to assess whether the logarithmic transformation improved residual behavior and overall fit.

Conclusion

The analysis began with weather-only models, which showed that snowfall was the most meaningful weather predictor, while temperature and precipitation variables were generally not statistically significant. However, these early models had relatively low explanatory power.

A snow-only model confirmed that snowfall mattered, but it still left much of the variation in incidents unexplained. A log-transformed version of that model slightly improved the structure of the relationship, suggesting that transformation could be useful.

After incorporating subway ridership and restricting the analysis to 2020 onward, model performance improved substantially. The linear model with snowfall and ridership showed that both predictors were statistically significant, indicating that both operational exposure and weather conditions contribute to MTA disruptions.

Finally, comparing the linear, log, and log-log models showed that the log-log specification provided the best overall fit according to AIC. The additional visualizations and diagnostics suggest that the transformed model better captures the relationship between incident counts, snowfall, and ridership. Overall, the results suggest that monthly MTA major incidents are better explained by a multiplicative relationship in which both system usage and weather intensity play important roles.